

Mechanising Denotational Semantics in Agda

Complete Code Listing

Peter D. Mosses^{a,1} Jesper Cockx^{a,2} Bernhard Reus^{b,3}

^a *Department of Computer Science
Delft University of Technology
Delft, Netherlands*

^b *School of Engineering and Informatics
University of Sussex
Brighton, United Kingdom*

Abstract

Formalisation of a semantic definition helps to ensure its wellformedness, and supports proofs about it. To minimise the effort required, formalisation should not involve significant reformulation or extension of the original definition. Here, we show how to formalise conventional Scott–Strachey denotational definitions by embedding them in Agda with almost no changes to λ -notation or domain equations.

We illustrate our approach with three examples: Scott’s D_∞ model of the untyped λ -calculus, Plotkin’s denotational semantics of PCF, and a semantics of a sublanguage of Scheme. Using appropriate postulates for domain constructors and their associated operations, the Agda type checker confirms the wellformedness of the embedded definitions. In previous work, such Agda formalisations have revealed several unsuspected wellformedness issues in published denotational definitions.

Some of our postulates are inconsistent with a classical set-theoretic interpretation of Agda. We conjecture that they would be consistent with an interpretation of Agda in the higher-order intuitionistic logic used by Alex Simpson in his work on synthetic domain theory.

Keywords: Denotational semantics, formalisation, mechanisation, Agda, postulates, synthetic domain theory

¹ Email: p.d.mosses@tudelft.nl

² Email: j.g.h.cockx@tudelft.nl

³ Email: bernhard@sussex.ac.uk

Contents

3	Postulated Domain Notation	3
3.1	Domains	3
3.2	Function domains	3
3.3	Recursive domains	4
3.4	Flat domains	4
3.5	Sum domains	5
3.6	Product domains	5
3.7	Updates	6
4	Illustrative Examples	7
4.1	Untyped λ -calculus	7
4.2	PCF	9
4.3	Scm: A Sublanguage of Scheme	11
5	Postulated Properties	15
6	Illustrative Tests	16
6.1	LC Tests	16
6.2	PCF Tests	17
6.3	Scm Tests	18
	References	19

3 Postulated Domain Notation

This section introduces the Agda module `Notation`,⁴ which declares conventional notation for domain constructors and the associated functions on their carrier sets.

3.1 Domains

Domains are elements of the Agda type `Domain`. A domain D is not itself a type, but it has a carrier type $\ll D \gg$, which always contains an element $\perp$. (Agda can usually infer the domains in λ -notation, so D is declared as an implicit argument of $\perp$.)

```
module Domains where
  postulate
    Domain : Set                -- Domain is the type of all domains
     $\ll\_ \gg$  : Domain  $\rightarrow$  Set    --  $\ll D \gg$  is the type of elements of  $D$ 
     $\perp$  : {D : Domain}  $\rightarrow$   $\ll D \gg$  --  $\perp\{D\}$  is the bottom element of  $D$ 
     $\mathbb{1}$  : Domain                --  $\mathbb{1}$  is a unit domain
```

In three previous papers based on denotational semantics in Agda, [3], [4], [5], the type of domains was defined by `Domain = Set`. However, postulating $\perp : D$ for all domains D was then *inconsistent* with the existence of an empty type in Agda. The current declaration `Domain : Set` circumvents that issue, but domains D now need to be distinguished from their carrier sets $\ll D \gg$.

The specified postulates that declare types and functions for domains are used only for type-checking denotational semantics in Agda. They do *not* define the conventional mathematical structure of domains, nor the algebraic and universal properties of the associated functions. Postulated equivalences between terms are used for testing denotations of terms; adding them as rewrite rules allows implicit use in proofs.

3.2 Function domains

The conventional notation in denotational definitions for the domain of all continuous functions from D to E is usually $D \rightarrow E$ or $[D \rightarrow E]$. However, Agda reserves the notation $D \rightarrow E$ for the *type* of *all* total functions from D to E . The following module declares the notation $D \rightarrow^c E$ for domains of continuous functions.

```
module Functions where
  postulate
     $\_ \rightarrow^c \_$  : Domain  $\rightarrow$  Domain  $\rightarrow$  Domain
    --  $D \rightarrow^c E$  is the domain of continuous functions from  $D$  to  $E$ 
```

In conventional denotational semantics, functions between domains are *automatically* continuous when defined in terms of λ -abstraction and application from primitive continuous functions associated with specific domain constructors. The carrier $\ll D \rightarrow^c E \gg$ of a function domain $D \rightarrow^c E$ should consist of just the (Scott-)continuous functions between the carriers $\ll D \gg$ and $\ll E \gg$. In Agda, however, that would require pairing all λ -abstractions with explicit proofs of their continuity, which is quite impractical – especially in continuation-passing style. To support type-checking direct use of conventional λ -notation for defining functions between domains, the type $\ll D \rightarrow^c E \gg$ is *rewritten* to the Agda type $\ll D \gg \rightarrow \ll E \gg$:

```
postulate
  dom-cts :  $\ll D \rightarrow^c E \gg \equiv (\ll D \gg \rightarrow \ll E \gg)$ 
  {-# REWRITE dom-cts #-}
```

⁴ In PDF, references to modules in the text are links to sections of the generated website that lists all the literate Agda modules [2].

Continuous *endofunctions* φ on D have (least) fixed points $\text{fix } \varphi$:

```
postulate
  fix : ⟨⟨ (D →c D) →c D ⟩⟩
  -- fix  $\varphi$  is the least fixed point of the continuous function  $\varphi$ 
```

Functions from an ordinary set A to a domain D form a domain $A \rightarrow^s D$:

```
postulate
  _→s_ : Set → Domain → Domain
  --  $A \rightarrow^s D$  is the domain of all functions from  $A$  to  $D$ 
```

The type $\langle\langle A \rightarrow^s D \rangle\rangle$ is *rewritten* to the Agda type $A \rightarrow \langle\langle D \rangle\rangle$:

```
postulate
  set-cts : ⟨⟨ A →s D ⟩⟩ ≡ (A → ⟨⟨ D ⟩⟩)
  {-# REWRITE set-cts #-}
```

3.3 Recursive domains

Conventional denotational semantics often involves groups of mutually recursive domain definitions. In Agda, recursive type definitions lead to non-termination of the type-checker. To avoid non-termination, it is sufficient to break the recursion by leaving (one or more) domains as *postulated*. The following operations can then be used to map values from a postulated domain to its structure and *vice versa* (as in [1]).

```
module Recursion where
  postulate
    _≅_ : Domain → Domain → Set
    -- an instance of  $D \cong E$  declares that the structure of  $D$  is the same as  $E$ 
    unfold : {{D ≅ E}} → ⟨⟨ D →c E ⟩⟩
    fold : {{D ≅ E}} → ⟨⟨ E →c D ⟩⟩
```

The *instance parameter* $\{\{D \cong E\}\}$ of the above operations restricts them to domains D and E such that $\text{instance } _ : D \cong E$ has been declared.

3.4 Flat domains

Lifting an ordinary set A by adding a $\perp$ element gives a flat domain, usually written $A_\perp$. The Agda module `Flat` postulates a corresponding domain constructor $A + \perp$, together with a function $\uparrow$ for injecting elements of A into $A + \perp$, and an operator $f^\#$ for extending a function f on A to a continuous function on $A + \perp$.

```
module Flat where
  postulate
    _+⊥ : Set → Domain
    --  $A + \perp$  constructs a flat domain
    ↑ : ⟨⟨ A →s (A + ⊥) ⟩⟩
    -- ( $\uparrow a$ ) injects  $a$  into  $A + \perp$ 
    _# : ⟨⟨ (A →s D) →c (A + ⊥) →c D ⟩⟩
    --  $f^\#$  extends  $f$  to map  $\perp$  to  $\perp$ 
```

3.4.1 Booleans

The McCarthy conditional operation $\beta \rightarrow \delta_1, \delta_2$ extends the usual ternary conditional choice to domains. It returns $\perp$ whenever its first argument is $\perp$.

```

module Booleans where
  Bool⊥ = Bool + ⊥
  _→_,_ : ⟨⟨ Bool⊥ →c D →c D →c D ⟩⟩ -- β → δ1 , δ2 is conditional choice
  _→_,_ = (λ b δ1 δ2 → if b then δ1 else δ2)#
    
```

3.5 Sum domains

The separated sum $D + E$ of two domains corresponds to lifting the disjoint union of their carrier sets. The following operations can be used directly for binary sums, and iterated for domains with more than two summands.

```

module Sums where
  postulate
    _+_ : Domain → Domain → Domain -- D + E is separated sum
    inj1 : ⟨⟨ D →c (D + E) ⟩⟩ -- inj1 δ is injection from D
    inj2 : ⟨⟨ E →c (D + E) ⟩⟩ -- inj2 ε is injection from E
    [_,_] : ⟨⟨ (D →c F) →c (E →c F) →c ((D + E) →c F) ⟩⟩
    -- [ φ , ψ ] applies φ to arguments in D, and ψ to arguments in E
    
```

Agda also supports type-checking the notation explained in §2.1. However, instead of defining the summands D of a separated sum domain E by an equation $E = \dots + D + \dots$, the domain E is merely *postulated*, and each summand is declared separately by instance $_ : E \gtrsim n \mapsto D$ (where n should be a different natural number for each summand). The inherently *dependent* types of the summand operations are as follows.

```

postulate
  _≳_↦_ : Domain → Nat → Domain → Set
  _in⊥_ : ⟨⟨ D ⟩⟩ → (E : Domain) → {E ≳ n ↦ D} → ⟨⟨ E ⟩⟩ -- δ in⊥ E injection
  _|⊥_ : ⟨⟨ E ⟩⟩ → (D : Domain) → {E ≳ n ↦ D} → ⟨⟨ D ⟩⟩ -- ε |⊥ D projection
  _∈⊥_ : ⟨⟨ E ⟩⟩ → (D : Domain) → {E ≳ n ↦ D} → ⟨⟨ Bool⊥ ⟩⟩ -- ε ∈⊥ D test
    
```

3.6 Product domains

The carrier of the binary cartesian product of two domains consists of all pairs of elements of the carriers of the argument domains. Neither the product nor pairing is associative. The following operations can be used directly for binary products, and iterated for products of more than two domains.

```

module Products where
  postulate
    _×_ : Domain → Domain → Domain -- D × E is cartesian product
    _,_ : ⟨⟨ D →c E →c (D × E) ⟩⟩ -- (δ , ε) is a pair of elements
    _↓1 : ⟨⟨ (D × E) →c D ⟩⟩ -- (δ , ε)↓1 is δ
    _↓2 : ⟨⟨ (D × E) →c E ⟩⟩ -- (δ , ε)↓2 is ε
    
```

The complete Agda code also declares notation for tuple domains $D \hat{\ } n$ and sequence domains $D \ast$ corresponding closely to that summarised in §2.1.

3.7 Updates

Environments $\rho : \langle\langle A \rightarrow^s D \rangle\rangle$ can be ‘updated’ (i.e., extended or overridden) using the conventional notation $\rho [\delta / a]$; stores $\sigma : \langle\langle (A + \perp) \rightarrow^c D \rangle\rangle$ use the notation $\sigma [\delta / \alpha] \perp$ for updates. The complete Agda code [\[2\]](#) defines these operations for any type A with an equality operation $== : A \rightarrow A \rightarrow \text{Bool}$.

4 Illustrative Examples

This section illustrates mechanisation of denotational semantics in Agda with three examples, all using the posulated [Notation](#) for domains and their associated operations:

- Untyped Lambda-Calculus [LC](#) (§4.1)
- Programming Computable Functions [PCF](#) (§4.2)
- A simple sublanguage [Scm](#) of Scheme [8] (§4.3)

4.1 Untyped λ -calculus

This section presents an Agda formalisation of a denotational semantics of the untyped λ -calculus. Elided module imports are shown in the complete code [2]. See the [Tests](#) module for some examples of abstract syntax terms and equivalence proofs.

4.1.1 Abstract Syntax

Abstract syntax is *not* regarded as a domain. In conventional Scott–Strachey style denotational semantics, abstract syntax is generally first-order: terms are finite, totally-defined elements.

A variable is written $x\ n$. The argument n merely distinguishes between variables – it is *not* a De Bruin index. The term constructor `var` below merely includes variables in terms.

In Agda, mixfix notation requires argument positions to be separated by characters other than spaces. Below, the constructors for function abstraction and application use the Unicode character $\sqcup$ as a separator.

```
module Examples.LC.Abstract-Syntax where
data Var : Set where
  x : Nat → Var
data Exp : Set where
  var _      : Var → Exp      -- variable reference
  (λ _ ⊔ _ ) : Var → Exp → Exp -- function abstraction
  (⊔ _ _ )   : Exp → Exp → Exp -- function application
```

4.1.2 Domain Equations

The postulate below declares `unfold` : $\langle\langle D_\infty \rightarrow^c (D_\infty \rightarrow^c D_\infty) \rangle\rangle$ and `fold` : $\langle\langle (D_\infty \rightarrow^c D_\infty) \rightarrow^c D_\infty \sim \rangle\rangle$, corresponding to a bijection between the domain D_∞ and the domain of continuous endofunctions on D_∞ . (Simply defining $D_\infty = (D_\infty \rightarrow^c D_\infty)$ would lead to non-termination of the Agda type-checker.)

```
module Examples.LC.Domain-Equations where
postulate
  D∞ : Domain -- corresponds to Scott's domain
  instance eqD∞ : D∞ ≅ (D∞ →c D∞) -- bijection
  Env = Var →s D∞ -- environments
```

Use of the conventional notation $\rho \ [\ \delta \ / \ v \]$ for updating an environment ρ to map v to d requires an equality test for variables [2].

4.1.3 Semantic Functions

The semantic equations below correspond closely to those found in textbooks on denotational semantics (e.g., [7]) apart from the need to wrap the domain of denotations in $\langle\langle \dots \rangle\rangle$.

```
module Examples.LC.Semantic-Functions where
```

$$\begin{aligned}
 \llbracket _ \rrbracket &: \text{Exp} \rightarrow \langle\langle \text{Env} \rightarrow^c \text{D}\infty \rangle\rangle \\
 \llbracket \text{var } v \rrbracket \rho &= \rho \, v \\
 \llbracket (\lambda v \sqcup e) \rrbracket \rho &= \text{fold} \, (\lambda \delta \rightarrow \llbracket e \rrbracket (\rho \, [\delta / v])) \\
 \llbracket (e_1 \sqcup e_2) \rrbracket \rho &= \text{unfold} \, (\llbracket e_1 \rrbracket \rho) \, (\llbracket e_2 \rrbracket \rho)
 \end{aligned}$$

4.2 PCF

PCF and its denotational semantics were originally defined by Dana Scott in 1969 [9] with combinators (S, K) instead of λ -abstraction. Gordon Plotkin subsequently defined a denotational semantics for PCF including λ -abstraction [6]. The Agda modules presented below formalise the latter definition.

PCF is an intrinsically typed language: every well-formed term has a unique type. The following grammar summarises the context-free abstract syntax of types σ, τ and terms M, N with variables α_i^σ ($i \geq 0$) and constants c . In §4.1.1 we reflect Plotkin’s presentation of PCF more accurately by exploiting Agda’s support for dependent types.

$$\sigma, \tau ::= \iota \mid o \mid (\sigma \rightarrow \tau) \quad (1)$$

$$c ::= tt \mid ff \mid \top \mid \mathbf{Y} \mid k_n \mid (+1) \mid (-1) \mid \mathbf{Z} \quad (2)$$

$$M, N ::= \alpha_i^\sigma \mid c \mid (M N) \mid (\lambda \alpha_i^\sigma M) \quad (3)$$

4.2.1 Abstract Syntax

The following abstract syntax of well-formed PCF terms in Agda uses indexed datatype definitions. PCF function types $\sigma \rightarrow \tau$ are written $\sigma \Rightarrow \tau$, and variables α_i^σ are written $\alpha \text{ i } \sigma$ (where the argument i merely distinguishes between variables – it is *not* a De Bruin index).

```

module Examples.PCF.Abstract-Syntax where
data Types : Set where
   $\iota$       : Types
   $o$        : Types
   $\_ \Rightarrow \_$  : Types  $\rightarrow$  Types  $\rightarrow$  Types
data Vars   : Types  $\rightarrow$  Set where
   $\alpha$     : Nat  $\rightarrow$  ( $\sigma$  : Types)  $\rightarrow$  Vars  $\sigma$ 
data  $\mathcal{L}^A$  : Types  $\rightarrow$  Set where
  tt      :  $\mathcal{L}^A$   $o$ 
  ff      :  $\mathcal{L}^A$   $o$ 
   $\top$      :  $\mathcal{L}^A$  ( $o \Rightarrow \sigma \Rightarrow \sigma \Rightarrow \sigma$ )
   $\mathbf{Y}$      :  $\mathcal{L}^A$  ( $(\sigma \Rightarrow \sigma) \Rightarrow \sigma$ )
   $k$        : Nat  $\rightarrow$   $\mathcal{L}^A$   $\iota$ 
   $(+1)$     :  $\mathcal{L}^A$  ( $\iota \Rightarrow \iota$ )
   $(-1)$     :  $\mathcal{L}^A$  ( $\iota \Rightarrow \iota$ )
   $\mathbf{Z}$      :  $\mathcal{L}^A$  ( $\iota \Rightarrow o$ )
data Terms : Types  $\rightarrow$  Set where
   $V\_$      : Vars  $\sigma \rightarrow$  Terms  $\sigma$ 
   $L\_$      :  $\mathcal{L}^A$   $\sigma \rightarrow$  Terms  $\sigma$ 
   $(\_ \_ \_)$   : Terms ( $\sigma \Rightarrow \tau$ )  $\rightarrow$  Terms  $\sigma \rightarrow$  Terms  $\tau$ 
   $(\lambda \_ \_)$  : Vars  $\sigma \rightarrow$  Terms  $\tau \rightarrow$  Terms ( $\sigma \Rightarrow \tau$ )
variable M N : Terms  $\sigma$ 
    
```

4.2.2 Domain Equations

The domains $\mathcal{D} \sigma$ form a “standard collection of domains for arithmetic” in PCF, written $\mathcal{D}_\sigma$ in [6]. As PCF is a simply-typed language, the domains $\mathcal{D} \sigma$ are not reflexive, so their formalisation in Agda can use ordinary type definitions, not involving bijections or embeddings.

```

module Examples.PCF.Domain-Equations where
D : Types → Domain          -- standard domains
D  $\iota$       = Nat $\perp$           -- natural numbers
D o         = Bool $\perp$        -- truth-values
D ( $\sigma \Rightarrow \tau$ ) = D  $\sigma \rightarrow^c$  D  $\tau$  -- functions
    
```

Environments ρ are type-preserving maps from variables to values. They are naturally modeled by a dependent type: $\text{Env } \sigma$ consists of type-preserving maps from variables in $\text{Vars } \sigma$ to their values in the domain $\mathcal{D } \sigma$. The environment $\rho \perp$ maps all variables to $\perp$.

```

Env = ( $\sigma$  : Types) →  $\langle\langle$  Vars  $\sigma \rightarrow^s$   $\mathcal{D } \sigma \rangle\rangle$  -- typed environments
 $\rho \perp$  : Env                                           -- initial environment
 $\rho \perp \_ \_ = \perp$ 
    
```

Extension or overriding environments requires instances of the equality tests for both variables and types. The definition of the latter [2] is somewhat tedious.

4.2.3 Semantic functions

The notation $\rho \llbracket \alpha \text{ i } \sigma \rrbracket$ gives the value of the variable $\alpha \text{ i } \sigma$ in ρ by applying $\rho \sigma$ to the variable.

```

module Examples.PCF.Semantic-Functions where
_ $\llbracket$  _  $\rrbracket$  : Env → Vars  $\sigma \rightarrow \langle\langle$   $\mathcal{D } \sigma \rangle\rangle$  -- typed variable denotations
 $\rho \llbracket \alpha \text{ i } \sigma \rrbracket = \rho \sigma (\alpha \text{ i } \sigma)$ 
    
```

The semantic function $\mathcal{A} \llbracket c \rrbracket$ gives the standard interpretation of the constant c . The corresponding definitions in [6] use case analysis on the domain $\mathcal{D } \iota$, which is not supported in our Agda formalisation (partly because it can express non-continuous functions).

```

 $\mathcal{A} \llbracket \_ \rrbracket$  :  $\mathcal{L}^A \sigma \rightarrow \langle\langle$   $\mathcal{D } \sigma \rangle\rangle$  -- typed constant denotations
 $\mathcal{A} \llbracket \text{tt} \rrbracket$  =  $\uparrow$  true
 $\mathcal{A} \llbracket \text{ff} \rrbracket$  =  $\uparrow$  false
 $\mathcal{A} \llbracket \supset \rrbracket$  =  $\lambda \beta \delta_1 \delta_2 \rightarrow (\beta \longrightarrow \delta_1, \delta_2)$ 
 $\mathcal{A} \llbracket Y \rrbracket$  = fix
 $\mathcal{A} \llbracket k \text{ n} \rrbracket$  =  $\uparrow$  n
 $\mathcal{A} \llbracket (+1) \rrbracket$  =  $(\lambda n \rightarrow \uparrow (n + 1))^\#$ 
 $\mathcal{A} \llbracket (-1) \rrbracket$  =  $(\lambda n \rightarrow \uparrow (n ==^N 0) \longrightarrow \perp, \uparrow (n - 1))^\#$ 
 $\mathcal{A} \llbracket Z \rrbracket$  =  $(\lambda n \rightarrow \uparrow (n ==^N 0))^\#$ 
    
```

The semantic function $\mathcal{A}' \llbracket M \rrbracket$ is written $\hat{\mathcal{A}} \llbracket M \rrbracket$ in [6]. It gives the denotation of the term M as a function of the environment ρ .

```

 $\mathcal{A}' \llbracket \_ \rrbracket$  : Terms  $\sigma \rightarrow \langle\langle$  Env  $\rightarrow^s$   $\mathcal{D } \sigma \rangle\rangle$  -- typed term denotations
 $\mathcal{A}' \llbracket V \alpha \text{ i } \sigma \rrbracket \rho$  =  $\rho \llbracket \alpha \text{ i } \sigma \rrbracket$ 
 $\mathcal{A}' \llbracket L c \rrbracket \rho$  =  $\mathcal{A} \llbracket c \rrbracket$ 
 $\mathcal{A}' \llbracket (M \sqcup N) \rrbracket \rho$  =  $\mathcal{A}' \llbracket M \rrbracket \rho (\mathcal{A}' \llbracket N \rrbracket \rho)$ 
 $\mathcal{A}' \llbracket (\lambda \alpha \text{ i } \sigma \sqcup M) \rrbracket \rho x$  =  $\mathcal{A}' \llbracket M \rrbracket (\rho [x / \alpha \text{ i } \sigma]')$ 
    
```

Comparison with Plotkin's original definition of PCF [6] confirms the directness of our Agda formalisation.

4.3 Scm: A Sublanguage of Scheme

Scm is a particularly basic sublanguage of the core Scheme expression language whose denotational semantics is defined in the Scheme reports [8]. The domains and auxiliary functions declared in this section are explained in the presentation of the conventional denotational semantics of Scm [4]; they involve the notation for sequences summarised in §2.1 of the present paper.

4.3.1 Abstract Syntax

The following grammar [4] summarises the abstract syntax of Scm expressions $E : \text{Exp}$ with integers $Z : \text{Int}$, constants $K : \text{Con}$ and identifiers $I : \text{Ide}$. The meta-variable $E^* : \text{Exp}^*$ implicitly ranges over arbitrary sequences of expressions.

$$K ::= Z \mid \#t \mid \#f \quad (4)$$

$$E ::= K \mid I \mid (E_0 \ E^*) \mid (\text{lambda } I \ E) \mid (\text{if } E_0 \ E_1 \ E_2) \mid (\text{set! } I \ E) \quad (5)$$

In the following Agda formalisation of the above grammar, the abstract syntax of sequences $E^* : \text{Exp}^*$ is made explicit: the empty sequence is represented by `⊔⊔⊔`, and sequence prefixing by `E ⊔⊔ E*`.

```

module Examples.Scm.AbstractSyntax where

Ide = String      -- identifiers

data Con  : Set where -- constants
  int     : Int → Con -- integer numerals
  #t      : Con       -- true
  #f      : Con       -- false

mutual

data Exp  : Set where -- expressions
  con     : Con → Exp -- constants
  ide     : Ide → Exp -- identifiers
  (|_ _ _) : Exp → Exp* → Exp -- procedure application
  (|lambda _ _ _) : Ide → Exp → Exp -- procedure abstraction
  (|if _ _ _ _) : Exp → Exp → Exp → Exp -- conditional choice
  (|set! _ _ _) : Ide → Exp → Exp -- assignment

data Exp* : Set where -- expression sequences
  []      : Exp* -- empty sequence
  _::_    : Exp → Exp* → Exp* -- sequence prefix

```

4.3.2 Domain Equations

The domains for Scm are somewhat simpler than for the denotational semantics in the Scheme standards [8], but still involve all our postulated domain constructors. Using definitional equations instead of postulated domain equivalences $D \cong E$ avoids the need for the functions `fold` and `unfold`.

```

module Examples.Scm.Domain-Equations where
postulate Loc : Set

L = Loc + ⊥           -- locations
N = Nat ⊥             -- natural numbers
T = Bool ⊥            -- booleans
R = Int + ⊥           -- numbers
P = L × L             -- pairs

```

```

U = Ide  $\rightarrow^s$  L                -- environments
data Misc : Set where
  null unallocated undefined unspecified : Misc
M = Misc  $\perp$                 -- miscellaneous
    
```

The conventional denotational semantics of Scm [4] defines the domain **E** of expression values by the equation $\mathbf{E} = \mathbf{T} + \mathbf{R} + \mathbf{P} + \mathbf{M} + \mathbf{F}$, and the domain **F** of procedure values by $\mathbf{F} = \mathbf{E}^* \rightarrow (\mathbf{E} \rightarrow \mathbf{C}) \rightarrow \mathbf{C}$. Mutually recursive groups of domain equations have well-defined solutions, but in Agda, defining both the corresponding domains **E** and **F** by equations would cause the type checker to diverge. Postulating one (or both) of these domains avoids divergence; postulating **E** also has the benefit that the embeddings and projections for its summands subsume the bijection between **E** and its intended structure.

```

postulate E : Domain          -- expressed values
S = L  $\rightarrow^c$  E                -- stores
postulate A : Domain          -- answers
C = S  $\rightarrow^c$  A                -- command continuations
F = E *  $\rightarrow^c$  (E  $\rightarrow^c$  C)  $\rightarrow^c$  C -- procedure values
    
```

The following postulates instantiate embedding (δ in $\perp$ **E**), inspection ($\varepsilon \in \perp$ **D**), and projection ($\varepsilon \mid \perp$ **D**) for each summand **D** of **E**.

```

postulate instance
  E += T : E  $\gtrsim$  1  $\mapsto$  T
  E += R : E  $\gtrsim$  2  $\mapsto$  R
  E += P : E  $\gtrsim$  3  $\mapsto$  P
  E += M : E  $\gtrsim$  4  $\mapsto$  M
  E += F : E  $\gtrsim$  5  $\mapsto$  F
    
```

4.3.3 Auxiliary Functions

The λ -notation in the Agda definitions of auxiliary functions for Scm corresponds closely to that in its conventional denotational semantics [4]. The main difference is the explicit application here of the function $\uparrow$ to inject elements of sets into the carriers of flat domains, cf. the definition of **truish** below.

```

module Examples.Scm.Auxiliary-Functions where

assign :  $\langle\langle$  L  $\rightarrow^c$  E  $\rightarrow^c$  C  $\rightarrow^c$  C  $\rangle\rangle$           -- assign  $\alpha \in$  stores  $\epsilon$  at location  $\alpha$ 
assign  $\alpha \in \theta$   $\sigma = \theta$  ( $\sigma$  [ $\epsilon / \alpha$ ]  $\perp$ )

hold :  $\langle\langle$  L  $\rightarrow^c$  (E  $\rightarrow^c$  C)  $\rightarrow^c$  C  $\rangle\rangle$           -- hold  $\alpha$  gives the value stored at  $\alpha$ 
hold  $\alpha \kappa \sigma = \kappa$  ( $\sigma$   $\alpha$ )  $\sigma$ 
    
```

In the continuation-passing style used to define auxiliary functions for Scm, giving explicit continuity proofs would be particularly tedious. For example, the function **hold** is simply a combination of λ -abstraction and application, which is wellknown to ensure continuity.

```

postulate new :  $\langle\langle$  (L  $\rightarrow^c$  C)  $\rightarrow^c$  C  $\rangle\rangle$           -- new gives an unallocated location

alloc :  $\langle\langle$  E  $\rightarrow^c$  (L  $\rightarrow^c$  C)  $\rightarrow^c$  C  $\rangle\rangle$           -- alloc  $\epsilon$  allocates a location for  $\epsilon$ 
alloc  $\epsilon \kappa = \text{new } (\lambda \alpha \rightarrow \text{assign } \alpha \epsilon (\kappa \alpha))$ 
    
```

```

truish : ⟨⟦ E →c T ⟧⟩ -- truish ε is true for all ε except false
truish ε = (ε ∈ ⊥ T) → (((ε | ⊥ T) == ⊥ ↑ false) → ↑ false , ↑ true) ,
           ↑ true
    
```

The remaining auxiliary function definitions shown here involve operations for (finite) sequences ϵ^* [2] that correspond to notation used in conventional denotational definitions:

- $\# \epsilon^*$ is the length of ϵ^* ;
- $\epsilon^* \downarrow n$ is its n th element (or $\perp$);
- $\epsilon^* \uparrow n$ is its n th tail;
- $\langle \rangle$ is the empty sequence; and
- $\langle \epsilon , \dots \rangle$ maps tuples $\epsilon , \dots$ to sequences.

```

cons : ⟨⟦ F ⟧⟩ -- cons ⟨ ε1 , ε2 ⟩ allocates and initialises a pair
cons ε* κ = (# ε* == ⊥ ↑ 2) →
            alloc (ε* ↓ 1) (λ α1 →
                alloc (ε* ↓ 2) (λ α2 → κ ((α1 , α2) in ⊥ E)) ,
            ⊥
    
```

In [4] the auxiliary function *list* is defined by recursion on ϵ^* . Agda accepts recursive definitions only when it can mechanically prove that the recursion terminates, which is not the case for arguments in postulated types. The following definition uses the postulated operation *fix* to avoid recursion.

```

list : ⟨⟦ F ⟧⟩ -- list ε* allocates and initialises a list
list = fix λ (list' : ⟨⟦ F ⟧⟩) → λ ε* κ →
        (# ε* == ⊥ ↑ 0) → κ (↑ null in ⊥ E) ,
        list' (ε* ↑ 1) (λ ε → cons ⟨ (ε* ↓ 1) , ε ⟩ κ)
    
```

4.3.4 Semantic Functions

The λ -notation in the Agda definitions of semantic functions for Scm corresponds closely to that in its conventional denotational semantics [4].

```

module Examples.Scm.Semantic-Functions where

K[ ] : ⟨⟦ Con →s E ⟧⟩ -- constant denotations
E[ ] : ⟨⟦ Exp →s U →c (E →c C) →c C ⟧⟩ -- expression denotations
E*[ ] : ⟨⟦ Exp* →s U →c (E* →c C) →c C ⟧⟩ -- sequence denotations

K[ int Z ] = ↑ Z in ⊥ E
K[ #t ] = ↑ true in ⊥ E
K[ #f ] = ↑ false in ⊥ E

E[ con K ] ρ κ =
    κ (K[ K ])
E[ ide l ] ρ κ =
    hold (ρ l) κ
E[ ( E ⊔ E* ) ] ρ κ =
    
```

$$\begin{aligned}
 & \mathcal{E}[\![\mathbf{E}]\!] \rho (\lambda \epsilon \rightarrow \mathcal{E}^*[\![\mathbf{E}^*]\!] \rho (\lambda \epsilon^* \rightarrow (\epsilon \mid \perp \mathbf{F}) \epsilon^* \kappa)) \\
 \mathcal{E}[\![(\text{lambda } l \sqcup \mathbf{E})]\!] \rho \kappa = & \\
 & \kappa ((\lambda \epsilon^* \kappa' \rightarrow \text{list } \epsilon^* (\lambda \epsilon \rightarrow \text{alloc } \epsilon (\lambda \alpha \rightarrow \mathcal{E}[\![\mathbf{E}]\!] (\rho [\alpha / l]) \kappa')))) \text{in} \perp \mathbf{E}) \\
 \mathcal{E}[\![(\text{if } \mathbf{E} \sqcup \mathbf{E}_1 \sqcup \mathbf{E}_2)]\!] \rho \kappa = & \\
 & \mathcal{E}[\![\mathbf{E}]\!] \rho (\lambda \epsilon \rightarrow \text{truish } \epsilon \longrightarrow \mathcal{E}[\![\mathbf{E}_1]\!] \rho \kappa , \mathcal{E}[\![\mathbf{E}_2]\!] \rho \kappa) \\
 \mathcal{E}[\![(\text{set! } l \sqcup \mathbf{E})]\!] \rho \kappa = & \\
 & \mathcal{E}[\![\mathbf{E}]\!] \rho (\lambda \epsilon \rightarrow \text{assign } (\rho l) \epsilon (\kappa (\uparrow \text{unspecified in} \perp \mathbf{E}))) \\
 \\
 \mathcal{E}^*[\![\sqcup \sqcup \sqcup]\!] \rho \kappa &= \kappa \langle \rangle \\
 \mathcal{E}^*[\![\mathbf{E} \sqcup \mathbf{E}^*]\!] \rho \kappa = \mathcal{E}[\![\mathbf{E}]\!] \rho (\lambda \epsilon \rightarrow \mathcal{E}^*[\![\mathbf{E}^*]\!] \rho (\lambda \epsilon^* \rightarrow \kappa (\langle \epsilon \rangle \S \epsilon^*))) &
 \end{aligned}$$

5 Postulated Properties

This module declares some properties of the conventional notation for Scott domains and the associated functions on their carrier sets.

The specified postulates that declare types and functions for domains are used only for type-checking denotational semantics in Agda. They do *not* define the conventional mathematical structure of domains, nor the algebraic and universal properties of the associated functions. Postulated equivalences between terms are used for testing denotations of terms; adding them as rewrite rules allows implicit use in proofs.

```

module Properties where
module Functions where
  postulate
    apply-fix : {φ : ⟨⟨ D →c D ⟩⟩} → fix φ ≡ φ (fix φ)
    -- apply-fix{φ} unfolds fix φ once
    {-# REWRITE apply-fix #-}
  module Recursion where
    postulate
      elim-unfold-fold : { { _ : D ≅ E } } → { e : ⟨⟨ E ⟩⟩ } → unfold (fold e) ≡ e
      {-# REWRITE elim-unfold-fold #-}
  module Flat where
    postulate
      elim-#-↑ : (f #) (↑ a') ≡ f a'
      elim-#-⊥ : (f #) ⊥ ≡ ⊥
      {-# REWRITE elim-#-↑ elim-#-⊥ #-}
  module Naturals where
    postulate
      elim-==⊥ : (↑ n1 ==⊥ ↑ n2) ≡ ↑ (n1 ==N n2)
      {-# REWRITE elim-==⊥ #-}
  module Sums where
    postulate
      elim-inj1 : [ φ , ψ ] (inj1 δ) ≡ φ δ
      elim-inj2 : [ φ , ψ ] (inj2 ε) ≡ ψ ε
      elim-[]-⊥ : [ φ , ψ ] ⊥ ≡ ⊥
      {-# REWRITE elim-inj1 elim-inj2 #-}
    postulate
      elim-∈⊥ : { { _ : E ≧ n ↦ D } } → { { _ : E ≧ n' ↦ D' } } → (δ : ⟨⟨ D ⟩⟩) →
        (δ in⊥ E) ∈⊥ D' ≡ ↑ (n ==N n')
      elim-|⊥ : { { _ : E ≧ n ↦ D } } → (δ : ⟨⟨ D ⟩⟩) → (δ in⊥ E) |⊥ D ≡ δ
      elim-∈⊥-⊥ : { { _ : E ≧ n ↦ D } } → { { _ : E ≧ n' ↦ D' } } → (δ : ⟨⟨ D ⟩⟩) →
        {n ≠ n'} → (δ in⊥ E) |⊥ D' ≡ ⊥
      {-# REWRITE elim-∈⊥ elim-|⊥ #-}
  module Products where
    postulate
      elim-↓1 : (δ , ε) ↓1 ≡ δ
      elim-↓2 : (δ , ε) ↓2 ≡ ε
      {-# REWRITE elim-↓1 elim-↓2 #-}

```

6 Illustrative Tests

6.1 LC Tests

```

module Tests.LC where
check-id : -- (λx1.x1)x42 = x42
  [ ( (λ x 1 ⊔ var x 1 ⊔ var x 42 ⊔ ) ) ] ≡ [ var x 42 ]
check-id = refl

check-const : -- (λx1.x42)x0 = x42
  [ ( (λ x 1 ⊔ var x 42 ⊔ var x 0 ⊔ ) ) ] ≡ [ var x 42 ]
check-const = refl

-- check-divergence : -- (λx0.x0 x0)(λx0.x0 x0) = ...
--   [ ( (λ x 0 ⊔ ( var x 0 ⊔ var x 0 ⊔ ) ) ⊔ (λ x 0 ⊔ ( var x 0 ⊔ var x 0 ⊔ ) ) ) ]
--   ≡ [ var x 42 ]
-- check-divergence = refl -- Agda type-checker diverges

check-convergence : -- (λx1.x42)((λx0.x0 x0)(λx0.x0 x0)) = x42
  [ ( (λ x 1 ⊔ var x 42 ⊔ ) ⊔
    ( (λ x 0 ⊔ ( var x 0 ⊔ var x 0 ⊔ ) ) ⊔ (λ x 0 ⊔ ( var x 0 ⊔ var x 0 ⊔ ) ) ) ) ]
  ≡ [ var x 42 ]
check-convergence = refl

check-abs : -- (λx1.x1)(λx1.x42) = λx2.x42
  [ ( (λ x 1 ⊔ var x 1 ⊔ ) ⊔ (λ x 1 ⊔ var x 42 ⊔ ) ) ] ≡ [ (λ x 1 ⊔ var x 42 ⊔ ) ]
check-abs = refl

check-free : -- (λx1.(λx42.x1)x2)x42 = x42
  [ ( (λ x 1 ⊔ ( (λ x 42 ⊔ var x 1 ⊔ ) ⊔ var x 2 ⊔ ) ) ⊔ var x 42 ⊔ ) ] ≡ [ var x 42 ]
check-free = refl
    
```

6.2 PCF Tests

```

module Tests.PCF where

-- Variables
e  =  $\alpha$  0  $\iota$ 
g  =  $\alpha$  1 ( $\iota \Rightarrow \iota$ )
h  =  $\alpha$  2 ( $\iota \Rightarrow \iota \Rightarrow \iota$ )
a  =  $\alpha$  3  $\iota$ 
b  =  $\alpha$  4  $\iota$ 

-- Arithmetic

check-41+1 :
   $\mathcal{A}' \llbracket () \ L \ (+1) \sqcup L \ k \ 41 \rrbracket \rho \perp \equiv \uparrow 42$ 
check-41+1 = refl

check-43-1 :
   $\mathcal{A}' \llbracket () \ L \ (-1) \sqcup L \ k \ 43 \rrbracket \rho \perp \equiv \uparrow 42$ 
check-43-1 = refl

-- Binding

check-id :
   $\mathcal{A}' \llbracket () \ (\lambda a \sqcup V \ a) \sqcup L \ k \ 42 \rrbracket \rho \perp \equiv \uparrow 42$ 
check-id = refl

check-k :
   $\mathcal{A}' \llbracket () \ (\lambda a \sqcup (\lambda b \sqcup V \ a) \sqcup L \ k \ 42) \sqcup L \ k \ 41 \rrbracket \rho \perp \equiv \uparrow 42$ 
check-k = refl

check-ki :
   $\mathcal{A}' \llbracket () \ (\lambda a \sqcup (\lambda b \sqcup V \ b) \sqcup L \ k \ 41) \sqcup L \ k \ 42 \rrbracket \rho \perp \equiv \uparrow 42$ 
check-ki = refl

check-suc-41 :
   $\mathcal{A}' \llbracket () \ (\lambda a \sqcup (L \ (+1) \sqcup V \ a) \sqcup L \ k \ 41 \rrbracket \rho \perp \equiv \uparrow 42$ 
check-suc-41 = refl

check-pred-42 :
   $\mathcal{A}' \llbracket () \ (\lambda a \sqcup (L \ (-1) \sqcup V \ a) \sqcup L \ k \ 43 \rrbracket \rho \perp \equiv \uparrow 42$ 
check-pred-42 = refl

check-if-zero :
   $\mathcal{A}' \llbracket () \ (L \ \supset \sqcup (L \ Z \sqcup L \ k \ 0) \sqcup L \ k \ 42) \sqcup L \ k \ 0 \rrbracket \rho \perp \equiv \uparrow 42$ 
check-if-zero = refl

check-if-nonzero :
```

```

 $\mathcal{A}' \llbracket () () L \supset () L Z \quad () L k 42 () () L k 0 () L k 42 () \rrbracket \rho \perp \equiv \uparrow 42$ 
check-if-nonzero = refl

-- Fixed points

check-fix-const :
 $\mathcal{A}' \llbracket () L Y () (\lambda e () L k 42 () ) \rrbracket \rho \perp \equiv \uparrow 42$ 
check-fix-const = refl

-- fix (λg. λa. 42) 2 ≡ 42
check-fix-lambda :
 $\mathcal{A}' \llbracket () () L Y () (\lambda g () (\lambda a () L k 42 () ) () L k 2 () ) \rrbracket \rho \perp \equiv \uparrow 42$ 
check-fix-lambda = refl

-- fix (λg. λa. ifz a then 42 else g (pred a)) 5 ≡ 42
check-countdown :
 $\mathcal{A}' \llbracket () () L Y () (\lambda g () (\lambda a ()$ 
 $\quad () () L \supset () L Z () V a () () L k 42 ()$ 
 $\quad () V g () L (-1) () V a () () () ()$ 
 $\quad () L k 5 () \rrbracket \rho \perp \equiv \uparrow 42$ 
check-countdown = refl

-- fix (λh. λa. λb. ifz a then b else h (pred a) (L (+1) b)) 4 38 ≡ 42
check-sum-42 :
 $\mathcal{A}' \llbracket () () () L Y () (\lambda h () (\lambda a () (\lambda b ()$ 
 $\quad () () L \supset () L Z \quad () V a () () V b ()$ 
 $\quad () V h () L (-1) () V a () () L (+1) () V b () () () ()$ 
 $\quad () L k 4 () L k 38 () \rrbracket \rho \perp \equiv \uparrow 42$ 
check-sum-42 = refl

```

6.3 Scm Tests

```

module Tests.Scm where

check-sum : {β : ⟨⟨ T ⟩⟩} → (β in ⊥ E) ∈ ⊥ T ≡ ↑ true
check-sum = refl

```

References

- [1] Abramsky, S. and A. Jung, *Domain theory*, pages 1–168, Oxford University Press, Inc., USA (1995), ISBN 019853762X.
<https://achimjungbham.github.io/pub/papers/handy1.pdf>
- [2] *Mechanising denotational semantics in Agda: Code* (2026).
<https://pdmosses.github.io/mfps2026-agda/>
- [3] Mosses, P. D., *Checking a denotational semantics of Scheme in Agda*, in: *Proceedings of the 26th ACM SIGPLAN International Workshop on Scheme and Functional Programming*, Scheme ’25, pages 3–15, Association for Computing Machinery, New York, NY, USA (2025), ISBN 9798400721625.
<https://doi.org/10.1145/3759537.3762694>
- [4] Mosses, P. D., *A compositional semantics for `eval` in Scheme*, in: *Proceedings of the Workshop Dedicated to Olivier Danvy on the Occasion of His 64th Birthday*, OLIVIERFEST ’25, pages 72–81, Association for Computing Machinery, New York, NY, USA (2025), ISBN 9798400721502.
<https://doi.org/10.1145/3759427.3760369>
- [5] Mosses, P. D., *Lightweight Agda formalization of denotational semantics*, in: F. Nordvall Forsberg, editor, *Proceedings of the 31st International Conference on Types for Proofs and Programs*, TYPES 2025, pages 286–290, University of Strathclyde, Glasgow, Scotland (2025).
https://msp.cis.strath.ac.uk/types2025/abstracts/TYPES2025_paper11.pdf
- [6] Plotkin, G. D., *LCF considered as a programming language*, Theoretical Computer Science **5**, pages 223–255 (1977), ISSN 0304-3975.
[https://doi.org/10.1016/0304-3975\(77\)90044-5](https://doi.org/10.1016/0304-3975(77)90044-5)
- [7] Reynolds, J. C., *Theories of Programming Languages*, Cambridge Univ. Press, Cambridge, UK (1998), ISBN 9780521106979.
<https://doi.org/10.1017/CB09780511626364>
- [8] *Scheme standards*.
<https://standards.scheme.org>
- [9] Scott, D. S., *A type-theoretical alternative to ISWIM, CUCH, OWHY*, Theoretical Computer Science **121**, pages 411–440 (1993), ISSN 0304-3975.
[https://doi.org/10.1016/0304-3975\(93\)90095-B](https://doi.org/10.1016/0304-3975(93)90095-B)